



软件测试

日期程序NextDay



2024-02-28 的下一天?

2023-02-28 呢?

2000-02-28 呢?

1900-02-28 呢?

2096-02-28 呢?

2100-02-28 呢?

**为什么一个只有几行代码的程序，
也会有这么多测试风险？**

日期程序 NextDay 需求规格说明

输入需要计算的日期判断用户输入的日期是否合法，包括年份是否规定范围，例如在 1-9999 之间、月份是否在 1-12 之间、日期是否在 1-31 之间。

计算下一天日期：根据输入的日期计算出下一天的日期。

NextDay 程序是三个 变量的函数：year、month、day，它返回输入日期之后的下一个日期：year、month、day 三个变量，规格条件如下所示。

① $1 \leq \text{year} \leq 9999$

② $1 \leq \text{month} \leq 12$

③ $1 \leq \text{day} \leq 31$

④ 输出日期为下一天

思考与讨论

需求规格的精确性

需求规格的完备性

隐含的需求规格

NextDay 程序中的复杂性主要来自于闰年判断。闰年的发现最早可以追溯到南北朝的数学家祖冲之。如果每四年计算一次闰年，则会出现微小错误。因此通过调整世纪年的闰年来解决。根据上述隐含需求规格，来实现具体的闰年判断。

闰年判断的子程序

一年的时间:

大约为365.2422天

闰年规则:

每四年一次，但世纪年需是 400 的倍数。示例：1980 年和1996 年是闰年，因为它们是4的倍数且不是世纪年。 2000 年是闰年，因为它是世纪年且是400的倍数，但 1900 年不是闰年，因为它是世纪年但不是400的倍数。

```
def LeapYear(year):  
    if year % 400 == 0:  
        return 1  
    if year % 100 == 0:  
        return 0  
    if year % 4 == 0:  
        return 1  
    else:  
        return 0
```

```
def LeapYear(year):  
    if year % 4 == 0:  
        if year % 400 == 0:  
            return 1  
        elif year % 100 == 0:  
            return 0  
        else:  
            return 1  
    else:  
        return 0
```

思考与讨论:

1. 上述两者写法哪个正确？尝试编写其他方式。
2. LeapYear函数的分支特征和Triangle的区别？


```
def next_day(year, month, day):
    tomorrow_month, tomorrow_day, tomorrow_year = month, day, year
    if month in [1, 3, 5, 7, 8, 10]:
        if day < 31:
            tomorrow_day = day + 1
        else:
            tomorrow_day = 1
            tomorrow_month = month + 1
    elif month in [4, 6, 9, 11]:
        if day < 30:
            tomorrow_day = day + 1
        else:
            tomorrow_day = 1
            tomorrow_month = month + 1
    elif month == 12:
        if day < 31:
            tomorrow_day = day + 1
        else:
            tomorrow_day = 1
            tomorrow_month = 1
            tomorrow_year = year + 1
```

```
elif month == 2:
    if day < 28:
        tomorrow_day = day + 1
    elif day == 28:
        if leap_year(year) == 1:
            tomorrow_day = 29
        else:
            tomorrow_day = 1
            tomorrow_month = 3
    elif day == 29:
        tomorrow_day = 1
        tomorrow_month = 3
    return (tomorrow_year, tomorrow_month, tomorrow_day)
```

思考与讨论:

1. 分支+计算程序的注意事项。
2. 尝试编写其他方法的日期计算。

普通日

月末

年末

2月28

2月29

闰年

非闰年

世纪年

400倍数世纪年

非法日期

测试设计示例

用例编号	year	month	day	预期输出	实际输出
1	2007	9	25	2007/9/26	
2	2010	5	3	2010/5/4	
3	2022	12	31	2023/1/1	
4	2001	2	28	2001/3/1	
5	2006	6	30	2006/7/1	
6	2001	2	29	无效日期	
7	2006	6	31	无效日期	

思考与讨论：边界是否考虑充分？

核心策略：聚焦边界值 + 闰年逻辑陷阱 + 非法输入组合，优先覆盖缺陷高发区（月末/年末、2 月天数、世纪年闰年规则、越界输入），舍弃普通中间值用例。

10 个高缺陷密度测试用例（输入：年，月，日 → 预期输出）

- 1. 2000, 2, 28 → 2000, 2, 29（验证世纪闰年 2 月 28→29）
- 2. 2000, 2, 29 → 2000, 3, 1（验证世纪闰年 2 月 29 合法且跨月）
- 3. 1900, 2, 28 → 1900, 3, 1（验证非闰年世纪年 2 月无 29 日，防误判闰年）
- 4. 2023, 2, 28 → 2023, 3, 1（验证普通平年 2 月 28→3 月 1 日）
- 5. 2023, 2, 29 → 无效/异常（验证平年 2 月 29 日被拒，防逻辑遗漏）
- 6. 2023, 4, 30 → 2023, 5, 1（验证小月 30 日跨月，防大月逻辑混用）
- 7. 2023, 1, 31 → 2023, 2, 1（验证大月 31 日跨月，防月份递增错误）
- 8. 2023, 12, 31 → 2024, 1, 1（验证年末跨年，防年份未递增）
- 9. 2023, 13, 15 → 无效/异常（验证月份上界越界，防范围检查缺失）
- 10. 2023, 2, 30 → 无效/异常（验证 2 月最大日越界，防硬编码 31 日逻辑）

注：若系统定义年份范围（如 1900—2050），可替换用例 9/10 为边界外年份（如 1899,6,15 或 2051,6,15）；若需更强健壮性，可将用例 10 改为 2023, 4, 31（小月超最大日）。所有“无效”用例应触发明确错误提示或异常，而非崩溃或错误日期。

- 1.缺少“普通闰年”
- 2.缺少“下界越界”
- 3.8月31日其实是比1月31日“缺陷发现能力更高”的测试。
- 4.9999, 12, 31 → ?

核心策略：聚焦边界值 + 闰年逻辑陷阱 + 非法输入组合，优先覆盖缺陷高发区（月末/年末、2月天数、世纪年闰年规则、越界输入），舍弃普通中间值用例。

10 个高缺陷密度测试用例（输入：年，月，日 → 预期输出）

- 1. 2000, 2, 28 → 2000, 2, 29（验证世纪闰年 2 月 28→29）
- 2. 2000, 2, 29 → 2000, 3, 1（验证世纪闰年 2 月 29 合法且跨月）
- 3. 1900, 2, 28 → 1900, 3, 1（验证非闰年世纪年 2 月无 29 日，防误判闰年）
- 4. 2023, 2, 28 → 2023, 3, 1（验证普通平年 2 月 28→3 月 1 日）
- 5. 2023, 2, 29 → 无效/异常（验证平年 2 月 29 日被拒，防逻辑遗漏）
- 6. 2023, 4, 30 → 2023, 5, 1（验证小月 30 日跨月，防大月逻辑混用）
- 7. 2023, 1, 31 → 2023, 2, 1（验证大月 31 日跨月，防月份递增错误）
- 8. 2023, 12, 31 → 2024, 1, 1（验证年末跨年，防年份未递增）
- 9. 2023, 13, 15 → 无效/异常（验证月份上界越界，防范围检查缺失）
- 10. 2023, 2, 30 → 无效/异常（验证 2 月最大日越界，防硬编码 31 日逻辑）

注：若系统定义年份范围（如 1900—2050），可替换用例 9/10 为边界外年份（如 1899,6,15 或 2051,6,15）；若需更强健壮性，可将用例 10 改为 2023, 4, 31（小月超最大日）。所有“无效”用例应触发明确错误提示或异常，而非崩溃或错误日期。

如果只允许 10 个，我会稍微重新组合

我建议用这组：

#	输入	预期	主要攻击目标
1	2024,2,28	2024,2,29	普通闰年
2	2000,2,28	2000,2,29	世纪闰年
3	1900,2,28	1900,3,1	世纪非闰年
4	2023,2,29	无效	平年非法闰日
5	2023,4,30	2023,5,1	30 天月
6	2023,8,31	2023,9,1	7/8 月特殊规律
7	2023,12,31	2024,1,1	跨年
8	2023,13,15	无效	月份上界
9	2023,0,15	无效	月份下界
10	2023,1,0	无效	日期下界

回顾与讨论

讨论：能否给定有限的测试确定一个NextDay程序？



测试设计与测试策略

- 分支与计算结合的测试
- 等价类与边界值的结合
- 控制流与数据流的结合

课后练习：如何测试农历与公历的转换程序？